TORQUE

FORCE · THROUGH LEVERAGE · TO A LIMIT

An AI agent that does real Salesforce work — on every org you run, production included.

Omid Mojtahedi · Salesforce implementation lead. Twenty-five-plus production orgs, mostly other people's. This is the method I already worked to, made enforceable so an agent has to work to it too.

Coding agents can already reach a Salesforce org. What they lack is the method a good admin has: describe before you reference, dry-run before you deploy, verify in the org rather than trusting the response, capture before-values so a bulk change can be undone, and treat a client's production org with the care it is owed.

Torque is that method, packaged — skills, rules, a validation harness, and enforcement that binds. It deploys and proves the deploy landed, updates in bulk with an undo trail, audits read-only, verifies in a real browser, and logs every change for handoff. The method is the same in a developer org and in a client's production instance; what changes is who authorises the write — production moves on an approval you issue, in one command, at your own terminal.

SPECIFICATION

Org credentials stored	none — sf CLI only	Licence	MIT · open source
Surface	Claude Code workspace	Org path	sf CLI + official MCP
Skills · subagents	5 · 2	Enforcement	2 hooks, 4 tool surfaces
Validation	120 checks, live org	Adversarial fixtures	257
Self-proof	19 mutators	Try it with no org	3 seconds

01

What it actually does

Asking an agent to “add a field” should mean the field exists, the right profiles can see it, and there is a record of the change — not that one API returned `200`. Each skill encodes the whole sequence, including the parts people skip when they’re tired.

DEPLOY, THEN PROVE IT

Dry-run → deploy → SOQL-verify the component exists → verify field-level security → confirm the field is gone on teardown. **No** custom field deployed through the Metadata API gets field-level security automatically — for any profile, System Administrator included — so it is invisible in layouts, reports and SOQL while the deploy reports success. Formula fields bite hardest: their permission entry has to be read-only, so the boilerplate that works everywhere else fails on them.

MASS-UPDATE WITH AN UNDO

Query current values, diff against intent, show counts, touch only records that actually change. Before-values are captured first, so the operation is reversible — and an empty diff stops rather than issuing a no-op write.

AUDIT AN ORG, READ-ONLY

A fixed survey — objects, active flows and triggers, validation rules, permission assignments, setup audit trail — dispatched to a subagent that carries no Edit or Write tools, so a large org doesn’t flood the main context.

RUN APEX UNDER REVIEW

Inline Apex is refused outright. You read the exact body at a terminal, approve it, and the gate re-hashes the file at run time — so a payload edited after approval fails on digest mismatch.

VERIFY IN THE BROWSER

Metadata deploying is not the page rendering. A real headless Chromium follows the frontdoor redirect — the one-time URL that turns a CLI session into a browser session — and asserts the Lightning shell came up — over a session token that never touches stdout.

KEEP A HANDOFF RECORD

Per-org session log: what changed, before → after, records touched by Id, what was verified, what still needs human UAT. Redacted — no PII, no raw rows, only what undo and the next session need.

WHO HOLDS THE SETTING

A torque wrench is not a weaker wrench. It is the one you reach for when the number matters — and that is the whole design. Sandbox and developer orgs are open ground. So is production — **the difference is that it opens on your authority rather than the agent’s.** One command at a terminal issues a single-use approval, or opens a bounded window for a session’s work. The agent asks, you decide, the work proceeds. What it cannot do is make that decision for you — and neither can a forgotten `--target-org`, which would otherwise fall through to whatever your default org happens to be.

Who it's for

Consultancies and admins running AI-assisted work in orgs they don't own, where the cost of a wrong write is somebody else's production data — and anyone who wants to use an agent on a real org without watching every keystroke. Torque adds no org access of its own: it runs on the Salesforce CLI and the official MCP server, and stores no org credentials.

POSITIONING

02

Why this isn't the MCP server

The official Salesforce MCP server and the `sf` CLI are good, and Torque runs on top of both. But they are a **tool surface**: they give an agent hands. They take no position on whether an operation is wise, whether it worked, or whether the org on the other end is a client's production instance. Filling that gap is the whole job.

	SALESFORCE MCP + SF CLI	GEARSET / COPADO	SFDX-HARDIS + CI LIBRARIES	TORQUE
Exposes org operations to an agent	yes	—	human / CI	**builds on**
Encodes *how* to do the work	—	UI-guided	for humans	**for agents**
Verifies the outcome in the org	—	partial	partial	**yes**
Distinguishes prod at write time	trusts alias	env model	trusts alias	**live query**
Carries platform knowledge that re-verifies itself	—	—	—	**44 entries**
Gets smarter from being used	—	—	—	**lessons → checks**
Shows what a change will set off, before it runs	—	design-time, in a UI	—	**at the command, in the agent loop**
Binds approval to a verified record count	command string	env / role	—	**record count**
Remembers what THIS org does	stateless	deploy history	—	**per-org, by orgId**
Proves its own checks can fail	—	—	—	**19 mutators**
Rollback across environments	—	**yes**	—	**single-org only**
Works outside Claude Code	**yes**	**yes**	**yes**	**enforcement yes, via the PATH shim; skills and rules no**
Vendor support / SLA	**yes**	**yes**	community	**none**

Torque loses the last three rows on purpose. It is a workspace layer, not a release-management platform: if you need environment-to-environment rollback or a vendor to call at 2am, buy the platform. What it does that they don't is govern an *agent* operating an org in real time.

Six things I could not find anywhere else

1 — It carries Salesforce knowledge that argues with itself. A tool surface gives an agent verbs. It does not tell it that a deployed field stays invisible until a PermissionSet grants it, that a bulk job can report success with thousands of failed

records, or that a no-op update rewrites who last touched forty thousand rows. Torque ships that as a structured catalogue — 44 entries, 10 of them re-verified against a live org on every release — where every one declares *how it is known*: re-checked against a live org, cited to Salesforce, or plainly marked as learned the hard way. The live ones are re-run by the harness, and a claim that stops holding fails the build.

THE ENTRY THAT WAS WRONG

One of them said that deploying a profile removes any permission the file does not mention — a common belief, and it was cited to Salesforce's own Metadata API guide. An adversarial review challenged it with an argument I could not answer: a retrieved profile contains permissions only for components in the package, so if absence removed things, every ordinary sparse deploy would strip the target org. That plainly does not happen.

So I stopped arguing and ran it. Two custom fields on a disposable Developer Edition org, both granted profile-level field access, then a profile deployed that mentioned only the second. **The first field's access survived.** A profile deploy is an overlay; the danger is what the file *says* — a full export carries explicit `readable=false` entries and overwrites exactly what it names.

The entry now states that, and the experiment *is* its verification: every release run deploys the fields, grants both, deploys the partial profile, asserts the survivor, and tears down. If Salesforce ever changes this, the build breaks. That is the whole design in one story — a knowledge base is only worth carrying if something can prove it wrong.

2 — The platform answers back, at the moment of the operation. Knowledge you have to remember to look up is knowledge you will not look up. The gates already parse every command, so the relevant entry prints as it goes past — including on a *refusal*, which is exactly when it matters, because that is when someone is deciding whether to override. Findings recorded against *this specific org* print first, because what is true here outranks what is true generally.

```
$ sf data update bulk --sobject Lead --file rows.csv --target-org acme-
sandbox

TORQUE PLATFORM NOTE [bulk-partial-failure] A bulk job can complete
successfully with
  thousands of failed records
  → Never treat a bulk exit code as evidence the data changed. Read the
failed-record
    results for the job id, and compare a before and after count.
TORQUE PLATFORM NOTE [unable-to-lock-row] UNABLE_TO_LOCK_ROW is usually
about a parent
  or a group, not the row you are writing
  → Order the batches so rows sharing a parent land together, or run
serially.
```

3 — It can tell you what an operation will set off, before it runs. "Update Type on every Prospect account" reads like one operation on N rows. It is N rows, plus every active trigger and record-triggered flow, plus every validation rule that must now pass on records saved years ago under different rules, plus the roll-ups that recalculate, plus — on a delete — the children that cascade and the lookup children left pointing at nothing. All of it is queryable. Nothing assembles it and puts it in front of you.

```
BLAST RADIUS — delete on Account @ acme-sandbox
  criteria : Type = 'Customer - Direct'
  scope    : 7 record(s)
  cascade  : 36 child record(s) deleted with the parent
             · Opportunity: 23
             · Contact: 13
  orphans  : 21 record(s) across 2 lookup child object(s) will survive the
delete
             and point at nothing
             · Case: 21
```

Deleting seven accounts strands twenty-one cases. Any source that cannot answer reports **UNDETERMINED** and the exit code changes, because a blast radius that quietly under-reports is worse than none — somebody would act on it. And an approval can be *bound* to that scope: a token approved for seven records is refused if the criteria come to match seven thousand.

4 — It treats "green but wrong" as the real failure mode. The expensive incidents are rarely errors; they are successes that were false. A deploy returns success and the field is invisible because formula fields receive no FLS. A flow reads Active in `FlowDefinitionView` and is absent from a metadata retrieve — on subscriber orgs that view can return zero rows for flows that are running. An agent reports a task complete because the tool said so. Every Torque operation ends by asking the *org* what is true, not by reading its own return code.

5 — It never trusts the alias. The CLI and MCP surfaces decide what org they are talking to by believing the string you passed. `--target-org acme-sandbox` can point anywhere, and the most likely production incident is a tired operator with the wrong alias in scrollback. Torque asks the org to identify itself at the moment of the write, and treats anything it cannot verify as production. It also refuses any command that changes the target and writes in the same breath:
`sf config set target-org=x && sf data update …` can re-point the write between the check and the execution, so the combination is denied.

6 — It is built to be disbelieved. Torque ships 257 adversarial fixtures and 19 mutation tests that neuter each guard in turn and require the attack it blocks to start working. A guard that cannot be shown to be load-bearing is decoration. Every enforcement claim carries a machine-checked label, resolved against the live code, so the documentation cannot quietly overstate it.

03

Setup — clone to working

Prerequisites: **Claude Code** (the hooks are its PreToolUse surface), `git`, `python3` ≥ 3.8, the Salesforce CLI ≥ 2.60, and any non-production org — a free Developer Edition is fine. `node` is optional and only affects the live browser check.

STEP 1 — SEE IT WORK BEFORE TRUSTING IT WITH AN ORG

No org, no credentials, no configuration. 24 real cases through the real hooks — 18 attacks the gate must refuse, 6 ordinary commands it must allow — in about three seconds.

```
git clone https://github.com/omoji-personal/torque && cd torque
python3 bin/torque-demo

Shell indirection — the gate never sees the literal word `sf`.
  DENIED  x=sf; $x data delete bulk --sobject Account --target-org acme-prod
          variable-assembled command → Salesforce operation
          hidden in a shell assignment value

Path expansion — only the secret's path once bash expands it.
  DENIED  cat /Users/you/.tor*/secret
          glob reaches the signing secret → reference to the
          trust anchor (~/.torque) — operator-only

Normal work is untouched. A gate that blocks real work gets switched off.
  allowed sf data query --query "SELECT Id FROM Account" \
            --target-org any-org
  allowed grep -rn 'sf data delete --target-org' .

  all 24 behaved correctly
```

STEP 2 — CONNECT AN ORG AND CONFIGURE

You authenticate; the agent never handles credentials. `torque init` refuses to continue if the org classifies as production, creates the trust anchor outside the repo, verifies the hooks are registered, and proves they bind before handing back control.

```
sf org login web --alias my-dev-org
python3 bin/torque init my-dev-org
```

STEP 3 — PROVE IT AGAINST YOUR OWN ORG

```
npm install          # optional — only the live browser check needs this
python3 harness/validate.py --profile release --target-org my-dev-org
```

Three profiles nest: `static` (99 checks, **no org needed** — the try-before-you-connect path), `capability` (117 checks, adding the live cycles), and `release` (120). `--only <check>` runs one.

STEP 3½ — OPTIONAL HARDENING, AND WHAT IT BUYS

Out of the box the gates run from `hooks/` in your clone. That is enough to stop the accidents and the enumerable circumventions, and it is what everything above describes. It leaves one honest gap: those files are in the repository, so anything that can write to the repository can rewrite the gate. Developing Torque itself needs exactly that — an operator-present *maintainer window* — and while one is open, the files being edited are the files adjudicating.

```
python3 bin/torque activate-enforcement   # runs static, then asks you to
type ACTIVATE
python3 bin/torque install-gates --project
```

That copies the *tested* tree into the trust anchor at mode 0400 behind an atomic symlink and points this workspace at it. A maintainer window still edits `hooks/`; the edit decides nothing until a present operator tests and promotes it. The second command writes an untracked local override, so your clone stays portable and its committed registration keeps working for everyone else. Reverse it with `install-gates --workspace`.

Two costs, stated rather than discovered: activation refuses on a dirty tree or a failing static profile, and while both registrations are present the gates may be consulted twice, so a gated write can take roughly double the usual 6.7–13.1 s.

STEP 4 — WORK

Open the folder in Claude Code. The hooks fire on the tool calls that can reach an org or the gate's own files — Bash, MCP, Edit/Write/MultiEdit and Read. Nothing further is needed. Start read-only — *"audit this org"* — and watch the survey come back. Skills are invoked in plain language: *"add a Priority Score formula field to Account," "set Industry to Technology for all Partner accounts," "log what we just did."*

> **SCOPE OF THE GATES**
>
> Hooks load from the **active workspace**, so they bind when you open this folder. A Claude Code session opened somewhere else loads no gates at all. If the agent will work in other repos, run `python3 bin/torque install-gates` — it registers them at user level with a recorded `TORQUE_HOME`, so they bind everywhere.

WHAT LANDS ON DISK

`~/.torque`	Trust anchor, mode 0700, **outside the repo** — signing secret, approval tokens, session grants, approved Apex bodies
`local/audit.log`	Every DENY and PROD-WRITE, and every ALLOW of a Salesforce operation — JSON lines, redacted, 0600
`local/sessions/`	Session log and undo data, 0600
`local/writable-orgs.json`	The write allowlist — a client can read exactly what the agent may touch

All of `local/` is gitignored, and a harness check fails if anything under it is tracked or the ignore rule is removed. The only network calls Torque makes are the `sf` CLI talking to your org, and — when you run the optional browser check — a headless Chromium following the frontdoor redirect into your own org. Nothing reports anywhere else.

THE MANUAL

04

The operations, worked through

safe-deploy — metadata that is verified, not just accepted

- **Describe first.** Referenced objects and fields are confirmed against the live org. Hallucinated API names are the most common cause of failed deploys; guessing is not permitted.
- **Dry-run** always precedes the real deploy.
- **Deploy.** The gate confirms the target is allowlisted *and* classifies as non-production from a live query at that moment.
- **SOQL-verify** the component is present — by querying the org, not by reading the deploy result.
- **FLS-verify.** Ship a PermissionSet with `fieldPermissions` alongside every custom field: the Metadata API grants none automatically, to any profile, System Administrator included. The field then exists but is absent from layouts, reports and SOQL — a silent failure the deploy status will happily call success. For a formula field the entry must be `editable=false`, or the deploy fails outright.
- **Functional check** (model-honoured): exercise it the way a user will meet it.

mass-update — a bulk change you can walk back

"Set Industry to Technology for all Accounts where Type is Partner." The refusal to act blindly is the feature — current values are queried, the exact Id set and per-field diff built, before-values captured, and only the records that actually differ are written:

```
matched 412 · already correct 118 · will change 294
```

Touching only real changes keeps audit history clean and avoids firing automation on records that were already correct. If the diff is empty the skill stops and issues nothing — a no-op bulk write still stamps `LastModifiedDate` and fires automation on records nobody changed. Undo refuses any record whose current value no longer matches what the run wrote.

The approval round-trip

This is the whole thesis in six lines:

```
you    "Delete the 400 test Leads in my-dev-org."

agent  sf data delete bulk --sobject Lead --file ids.csv \
         --target-org my-dev-org

TORQUE GATE DENY [destructive_data_gate] bulk-delete requires
       operator-present approval. An agent cannot mint an
       HMAC-signed token. Run: python3 ~/torque/bin/torque
       approve 00D…UAG bulk-delete

you    $ python3 ~/torque/bin/torque approve 00D…UAG \
         bulk-delete
       Approve bulk-delete on org 00D…UAG? [type YES]: YES
       approved: single-use token for bulk-delete on
       00D…UAG (5 min)

agent  (retries — token consumed, delete runs; a second attempt is denied)
```

The gate prints the resolved path rather than a bare `torque`, because nothing puts this repo's `bin/` on your `PATH` — a remediation you cannot paste is not a remediation. Add it to `PATH` yourself if you prefer the short form.

Operations that need approval: `apex`, `bulk-delete`, `bulk-write`, `where-delete`, `where-update`, `destructive-metadata`, `write`. Tokens live five minutes and are single-use. For production, add `--prod` and type `WRITE PRODUCTION`; for a window, `--session <minutes>` (≤ 120, revocable with `--end-session`).

Anonymous Apex — the strictest path

Inline Apex is refused outright: a body passed on the command line or stdin cannot be bound to an approval. Instead the body lives in a file, `torque approve --apex-file` prints it *in full* at your terminal — never truncated, because approving code you only half saw is not approval — and writes a read-only copy named for its own SHA-256. The gate re-hashes that file at run time, so an Apex payload edited after approval fails on digest mismatch. Apex routed through MCP is refused entirely and redirected to the file path.

blast-radius — what it will set off, before it runs

Read-only, and it runs before the operation rather than after. It reports scope, the active triggers and record-triggered flows the operation can fire, the validation rules that must now pass on records saved years ago under different rules, the roll-ups that recalculate, and — on a delete — the children that cascade and the lookup children left pointing at nothing.

```
python3 bin/torque blast-radius --target-org my-org --sobject Account \
    --operation delete --where "Type = 'Customer - Direct'"
```

Its one defining property is that **"none" and "could not tell" are different answers**. Any source that fails reports UNDETERMINED and the exit code changes. A blast radius that quietly under-reports is worse than none at all, because somebody would act on it — so the harness points the tool at an unreachable org and requires

UNDETERMINED for every part. Three categories are kept apart that would otherwise read alike: platform views and entity types that refuse `COUNT()` (a property, not a gap), self-referencing lookups where Salesforce forbids a semi-join on the same object, and genuine unknowns.

checkup — which of the platform's traps are live in this org

twelve catalogue entries carry a probe: the query that answers "is this happening to me right now". `checkup` runs them and reports what it found, each finding citing the entry that predicted it and how that entry is known. That citation is the difference between this and an org health scanner — the scanners tell you what they found and cannot tell you how they know.

```
CHECKUP — my-org   12 probe(s) from the platform catalogue

  LIVE IN THIS ORG — the probe returned rows
    [deleted-field-rename]  3 row(s)
      A deleted custom field is renamed with `_del`, and the name is freed
exactly once
    [recycle-bin-retention]  6 row(s)
      A deleted record is recoverable for up to 15 days

  CONFIRMED — the platform refused the query, exactly as the entry says
    [self-join-forbidden]  MALFORMED_QUERY: The inner and outer selects
should not be on
      the same table
```

It does not execute the catalogue. Shelling out a probe string would make a text file a command-execution surface, one careless entry away from running anything; only SOQL is extracted and run read-only, and anything else is reported as **not executed** with the reason. Running these for the first time found two entries that had been unrunnable since they were written — nobody could know, because nothing had ever asked. A probe that cannot run now fails the build, because `checkup` would otherwise report a clean org on a trap it never tested.

audit-org, session-log, orgs

audit-org runs a fixed read-only battery through the `org-explorer` subagent, which carries no Edit or Write tools and whose tool list is machine-checked by the harness. **session-log** (`torque log`) appends a redacted, 0600 entry with before → after values and computes a `reversible` flag, warning when there is no before-value to reverse to. **orgs** handles connection, live classification, and the write allowlist — connecting is `sf org login web`; deciding whether Torque may *write* is an explicit per-org operator decision.

The rules the agent carries

`org-safety`	Production read-only by default; classification precedence; delete test records by Id only, never by title or date match
`live-verification`	Never guess an API name; describe before you reference; namespace resolution in managed-package orgs
`deployment`	Deploy order, formula-field FLS, dry-run discipline, post-deploy verification, the anti-pattern list
`platform-quirks`	A catalogue of platform behaviour that silently produces wrong answers. The entries are assertions from practice; the dated ones carry a reproduction in a detail file, and one is confirmed against a named org
`browser-testing`	Frontdoor auth, Lightning timing, shadow DOM, never wait on network-idle
`working-discipline`	Session logs, privacy posture, QA honesty — name exactly which layers ran
`validation`	Nothing ships without the harness green against a live org

THE CATALOGUE — AND THE ONE THAT WAS WRONG

Forty-four entries across nine domains: ten re-verified against a live org on every release run, seventeen cited to Salesforce, seventeen marked plainly as learned the hard way. The label is load-bearing rather than decorative — every one of the ten live verifiers is fed, by the harness, the org response that would make its entry false, and is required to say so. Three of them previously could not fail at all: they queried the org, ignored the answer, and returned true. That is fixed, and the honest number went *down* as a result.

So the shape is visible rather than asserted:

2026-08-01	A deleted custom field sits 15 days in the deleted-fields queue and keeps counting against the object's field limit. Salesforce **renames it with a `_del` suffix**, which frees the original API name — but only once: a second delete of the same name finds `X_del` taken, leaves the field under its original name, and only *then* blocks reuse. Verified by counting the queue on the validation org, not by inferring it from a collision.
2026-08-01	`FlowDefinitionView` is a *standard*-API object: query it through the Tooling API and it errors, which some clients surface as zero rows — the usual reason someone concludes the view is unreliable. And do not verify activation by grepping `<status>` from a retrieve: since API v44 a retrieve returns only the latest version, so an active v3 reads Draft the moment someone clones it. Ask `FlowDefinition.ActiveVersionId` (Tooling) or `FlowDefinitionView.IsActive` (standard).
2026-07-20	Phishing-resistant MFA enforcement (production from 20 July, staggered; the widely-quoted 1 July date was rolled back) restricts *privileged* users to FIDO2/WebAuthn. The change that breaks automation is that **TOTP no longer qualifies** — a shared secret could be fed to a script, an authenticator cannot. It does not touch the OAuth session the `sf` CLI already holds, which is why browser verification hands off from that session rather than logging in.

The same discipline covers namespace resolution: the same object is `Thing__c` in the source repo and `ns__Thing__c` in a subscriber org, so an API name is resolved against the org it will run in, never against the package it came from.

Each rule is labelled with what actually backs it — a hook, a harness check or the model's judgment. Only `platform-quirks` is the last of those, and it says so. The rule files are also byte-capped and the cap is enforced, which is why they read terse: context spent on a rule is context not spent on the work.

SAFETY MODEL

05

How production is authorised

L1 **Credentials.** Connect production read-only. Torque stores no org credentials — the `sf` CLI is the only credential path. This is the real boundary and the honest answer to "what if everything else fails."

L2 **Authorization by identity.** A write is allowed only when its target is on an explicit allowlist *and* classifies as non-production from a live `Organization` query at that moment. A Developer Edition org reports `IsSandbox=False` and is still not production, which is why the verdict has three values and not two — and why `IsSandbox` is checked as a real boolean, since a malformed `"false"` string is truthy in Python and would flip production to sandbox. Scratch-org evidence matches on org Id only, because alias equality is forgeable. The classification cache is never consulted on the authorization path, because the agent can write to it.

L3 **Deterministic gates.** Two PreToolUse hooks, registered across Bash, MCP, Edit/Write and Read, share one expansion-aware parser that authorizes by *parsing argv* and defaults to deny. Every MCP tool call goes through the same gates: an org-touching tool whose verb is unrecognized is treated as a write, and names are decomposed across `_`, `-`, camelCase and acronym boundaries so `bulkDeleteRecords` and `HTTPDelete` both classify.

L4 **Production approval.** Real work includes deploying to a client's production org, and Torque does it — the approval is simply yours to give rather than the agent's to assume. `torque approve <org> <op> --prod` mints a single-use token; `--session <minutes>` opens a bounded, revocable window. Every production write is audited as a `PROD-WRITE` line in `local/audit.log`.

L5 **Verified enforcement.** Every rule is labelled hook-enforced, harness-enforced or model-honoured, and the labels resolve against the live hook set and check registry. The first version of that checker validated only the hook labels — so a `harness-enforced` label naming a check that had never existed passed silently. It was caught by the discipline it enforces.

AN APPROVAL CAN BE BOUND TO A RECORD COUNT, NOT JUST A COMMAND

Approval everywhere else is per-command-string or per-session: the operator vouches for their own reading of a `WHERE` clause, and nothing checks it again. But data moves. Criteria that matched seven rows when the operator looked can match seven thousand by the time the command runs, and no command string can express the difference — which is why nobody's approval does.

`approve … --impact --sobject Account --where "…"` computes the blast radius, shows it, establishes the count, and signs that count into the token. At the gate the count is re-established and the token refused if the operation grew:

```
TORQUE GATE DENY [destructive_data_gate] approved for 2 Lead record(s), the
criteria now match 8.
The operation grew after approval; re-approve with the current scope.
```

A count that cannot be re-established refuses rather than assuming it is unchanged: an impact-bound token whose impact is unknown is not a token. This is opt-in and additive — without `--impact` the ordinary path is unchanged, because a gate that makes routine work harder is a gate people switch off.

What this is not. It is a ceiling, not a proof of scope. A different set of the same or smaller size satisfies it, and rows can change between the count and the DML — signing record Ids would narrow the first gap and not close the second, because that race lives in the platform rather than here. The guarantee is precisely this: the operation cannot affect more rows than were approved.

FAILING CLOSED HAS TO SURVIVE THE HOST, NOT JUST THE CODE

A hook that Claude Code kills for running long exits at a status that means *allow*. So bounding each `sf` callout was not enough: classification makes two sequential calls, and two 45-second caps come to 90 seconds — past the host's timeout, and therefore a fail-open window. Torque sets one wall-clock budget for the whole gate, clamps every callout to what remains, and treats an exhausted budget as an *unverifiable* org — which denies, but is reported as unverifiable rather than as production, because "I could not establish what this org is" is not a finding that it is production. A hung `sf` on the PATH is a live test case in the harness.

WHY THE AGENT CANNOT SELF-APPROVE

The gate runs *before* bash expands, so it judges what a token *could* become, not what it says. Every argument is converted to an absolute glob pattern; inline `VAR=` assignments are resolved in order; undefined variables are treated as bash treats them; command substitution becomes a wildcard; and matching uses a dynamic-programming glob-prefix matcher, because an earlier positional compare misaligned on `**`. Four escalating attacks — `~/.torq*/secret`, `a=.tor;b=que;cat ~/$a$b/secret`, a path assembled across three variables, and `/Users/**/…/.[t]orque/sec[r]et` — all die on that one property. Tokens are HMAC-signed with a secret the agent's Bash, Edit *and* Read tools are all denied; the issuer refuses unless attached to a real login TTY clear of the agent; and a token is claimed by an atomic rename *before* it is read, so two concurrent gates cannot both spend it.

> **A FLOOR NO TOKEN LIFTS**
>
> A short list of sObjects is shielded from bulk-delete shapes on **every** org, approved or not — `harness/checks/protected-objects`, seeded with Account, Contact and Opportunity. It is a floor, not an authorization check, and the shield reaches inside an approved Apex body too. Edit it yourself: the agent's Edit and Write tools are denied on it, as are Bash write shapes.

THREAT MODEL — WHERE THE BOUNDARY HONESTLY SITS

The gates stop two categories. **Accidents, with certainty**: a wrong alias, a forgotten `--target-org` that would silently use the default org, a fat-fingered bulk delete. This is the dominant real-world risk and it is closed structurally. **Enumerable circumvention** through the tool surface: indirection, grouping, wrapper runners, `eval` and here-strings, `xargs` stdin, legacy `force:data:*` verbs, glued redirects, `cd`-desync aimed at the gate's own files, forged tokens, and destructive operations routed through MCP under a non-obvious name. Each has a named fixture that must deny.

What no PreToolUse hook can stop is a same-user actor who steps *outside* the tool surface: writing a script and running it, so `sf` executes as a subprocess the hook never sees; forging a login session to fake operator presence; or reading the anchor by OS means the gated tools never touch. Those sit at the filesystem and credential boundary, not somewhere a hook adjudicates. For them the load-bearing defence is L1 — connect production read-only, and never leave a production org authenticated in an autonomous session. **A write that fully bypasses the gate meets nothing else: the live non-production check runs inside the gate, so there is no second layer behind it.** That is precisely why L1 is load-bearing here rather than belt-and-braces — this sentence used to claim the opposite, and an outside review was right that it could not be true of a check the bypassed component performs. A PATH shim closes the subprocess channel and **is shipped** — `torque install-gates --shim`. It is not an extra: on 2026-08-05 an agent wrote a Salesforce write into a file, ran it with bash, and deployed to an org that was not on the write allowlist — no hook fired and no audit line was written. **Without the shim the allowlist is advisory rather than enforced.** This page called it roadmap while the README said it shipped; two files in one repository disagreeing about whether a control exists is worse than either answer.

TROUBLESHOOTING

06

When it refuses

Denials print as `TORQUE GATE DENY [hook] reason`, and every decision — allow and deny alike — lands in `local/audit.log`. Common symptoms:

`Nothing is gated`	The session isn't in this workspace. Hooks load per-workspace — run `torque install-gates` for user-level registration.
`init refuses my org`	Working as designed: it will not configure against an org that classifies as production, and an org it cannot classify is *unverifiable* — refused for the same reason and named differently, so nobody approves a production override for a sandbox. Use a Developer Edition org.
`Everything says PRODUCTION`	Usually expired `sf` auth — classification fails and the verdict is *unverifiable*, which denies and explicitly says it is not a production finding. Re-authenticate. Correct fail-safe, confusing symptom.
`Denied despite a token`	Either the token expired (5 min), the `<op>` didn't match the operation, or the sObject is on the protected list — which no token lifts.
`clean_ip reports N/A`	Expected on any clone but the author's: it needs a private pattern list. An honest N/A, not a failure, and it does not degrade the verdict.
`browser_render BLOCKED`	No Chromium — `npx playwright install chromium`. Detection is macOS-path-based today.
`A denial you believe is wrong`	`torque approve <org> --session 30` unblocks you for a bounded window. Then please report it — a false denial is a bug worth having; see `SECURITY.md`.

EVIDENCE

07

Proving it — the validation harness

A LIVE CAPABILITY CYCLE

Deploy a custom field to your org → verify it exists by SOQL → verify field-level security → delete it with `purgeOnDelete` → confirm no live field remains under that API name, and report the `_del` tombstone Salesforce keeps for 15 days rather than pretending it is not there. Then a mass-update with a working undo, and a real headless Lightning render. Nothing is mocked; if the org disagrees, the check fails.

EXIT CODES ARE NOT PROTECTION

An exit code proves a hook returned deny. It does not prove the org was protected. `harness/tests/live_matrix.sh <org>` closes that gap: it gates each operation, *executes* the allowed ones for real, and re-queries the org after every one — data untouched after a denial, the record actually gone after an approved delete, residue zero at teardown. It also puts a hanging stub `sf` on the PATH to prove the gate budget denies rather than hangs.

257 ADVERSARIAL FIXTURES

Every attack class found across the audit rounds, each a named, runnable regression test. **67 of them — 28% — assert that *ordinary work is allowed*,** which is the property that keeps a gate switched on. That share has been climbing on purpose: a replay of six months of real client commands found a parser defect that refused 71% of them, and 193 fixtures had not caught it because their allow cases were toy commands. Proving a control refuses is not proving it accepts.

THE KNOWLEDGE IS MUTATION-TESTED TOO

Mutation testing is applied to the guards and, since an external review caught it, to the *knowledge*. Each live-claim verifier is fed the org response that makes its entry false and is required to return false. Three of the original five could not fail at all — they queried the org, ignored the answer, and returned true — while the summary line read "5/5 live claims re-verified". The label meant nothing until something tried to break it.

The same review challenged an entry's *content*, and it turned out to be wrong. The correction, and the experiment that now re-runs on every release to keep it honest, is the story in section 02. Both are the same discipline: a claim nobody can falsify is not knowledge, it is decoration.

19 MUTATION TESTS — THE PART THAT MATTERS

Each temporarily neuters one guard and requires the corresponding attack to succeed — or, for the static scanners, requires the check to fail. One needs an org, because the check it neuters queries one. Three exercise the clean_ip scan against a private pattern list and report as operator-only on your clone rather than pretending to pass. The rest run anywhere.

WHO GUARDS THE GUARDS

An honest question this project had the wrong answer to for weeks. Developing Torque means editing Torque's own gates, which needs an operator-present *maintainer window* — and while that window was open, the files being edited were the files adjudicating every write. A window granted to fix a check also granted the power to delete one. That is not theoretical: a window-legal edit making `authorize_write` return `(True, "mutant")` turned a production denial into an allow on the very next tool call.

`torque activate-enforcement` separates the two. It runs the static profile, refuses on a dirty tree or a failing one, and copies the tested tree into the trust anchor at mode 0400 behind an atomic symlink. From then on a maintainer window still edits `hooks/` — and the edit decides nothing until a present operator tests and promotes it.

The check that measures this reported **N/A for as long as the feature was unused**, because there was nothing to measure and an empty loop is not evidence. It now reads: *a window-legal edit to `authorize_write` flips the workspace copy (exit 0) and leaves the active gate denying*. That is the difference between a safety property being configured and being measured, and it is the only version worth claiming.

```
python3 harness/validate.py --self-test

  · clean_ip mutators (3): operator-only — the private pattern list is not
    published, so they are skipped rather than failed
  ✓ secret_scan mutator: expected FAIL, got FAIL
  ✓ anchor-guard mutator: expected NOT-deny, got exit 0
  ✓ destructive token mutator: expected ALLOW, got exit 0
  ✓ redirect-detection mutator: expected NOT-deny, got exit 0
  ✓ wrapper (wrapped_sf) mutator: expected NOT-deny, got exit 0
  ✓ expansion-awareness mutator: expected NOT-deny, got exit 0
  ✓ glob-matcher (_glob_reaches) mutator: expected NOT-deny, got exit 0
  ✓ command-word (cmd_base) mutator: expected NOT-deny from either gate, got
exits 0/0
  ✓ deadline (_arm_deadline) mutator: expected FAIL, got FAIL
  ✓ single-use (rename-claim) mutator: expected FAIL, got FAIL
  ✓ identity-binding mutator: expected FAIL, got FAIL
  · redaction mutator: needs --target-org (the session-log check queries an
    org) — skipped rather than counted

  self-test: all runnable mutators caught — NOT RUN: clean_ip ×3
  (operator-only); redaction (needs --target-org)
```

That last line is the design. A skip is never allowed to read as a pass — an earlier version printed "ALL MUTATORS CAUGHT" while silently omitting the org-bound mutator, which is exactly the false green this harness exists to prevent. It was found by auditing this guide.

THE HARNESS'S OWN FAILURE MODES

Three, each discovered the hard way and left in the code as comments. Mutators spawn `validate.py` as a subprocess, and the child re-mutated the same file — observed for real as stacked MUTANT lines in `lib.py`, now prevented by a recursion guard. The historical-blob mutator plants through git plumbing onto a throwaway ref, because an earlier `--hard` version destroyed uncommitted work. And the redaction mutator patches the live module object rather than the file, because the module is already imported and a file edit would leave the cached function in place and report a **false pass** — a mutation test that would have proven nothing.

The dated run log is `harness/VALIDATION.md`. It is meant to be reproduced, not believed.

PROVENANCE

08

Where it came from

The method is years old; this repository is days old. If you check the git history you will find a short one, and that is the honest shape of it: I learned this method across more than twenty-five live Salesforce orgs, mostly other people's production, and extracting it into something portable was the fast part. What took the time is logged per-commit in `harness/VALIDATION.md`.

01 **The pattern.** Agents were genuinely useful in orgs, and the near-misses were never exotic. Wrong alias. A deploy that reported success and changed nothing a user could see. A bulk update with no way back. Prompt instructions — "be careful, don't touch prod" — do not bind a model under load.

02 **The insight.** The fix isn't a better prompt, it's a layer that doesn't negotiate. Enforcement has to live outside the model's context, parse what is actually being run, and default to deny — because the model that is confused is the one asking.

03 **The design, audited before it was built.** Nine rounds of multi-model adversarial review against the plan — independent models arguing to falsify the design, every finding required to carry a runnable command that reproduced the flaw — before a line of code existed.

04 **The build, attacked after.** Then four adversarial rounds against the working code, including a reviewer whose job was to run each exploit against a real `sf` CLI rather than reason about it. Every finding closed with a general fix and a named fixture — parsing argv instead of matching text, judging what a glob could expand to, failing closed on any indirection it can't resolve. Convergence was judged on the severity of findings rather than their count: the rounds stopped when every new finding was a refinement of a known front rather than a new architectural class.

05 **And again on this guide.** Four more lenses — fact-verification, design, a hostile buyer and completeness — were run against the document you are reading. They found three fabricated behaviours, a command the deny messages told you to run that did not exist, and the false green in the self-test above. All fixed; that's what the process is for.

06 **How it was built, since a short history invites the question.** With a coding agent, under these gates, in days — which is the method rather than a caveat. What twenty-five-plus orgs taught is not the typing: it is knowing which failure is worth a gate, refusing to claim what the harness cannot prove, and recognizing when a review round has stopped finding architecture and started finding refinements. Anyone hiring for AI-assisted delivery should want to see that seam, so it is stated rather than left to be inferred.

Reference

COMMANDS — python3 bin/torque <command>

`demo`	24 real cases through the real hooks. No org, no credentials, ~3s.
`init <org>`	Clone → configured. Refuses production; creates the trust anchor; verifies the gates bind.
`approve <org> <op>`	Mint a single-use token (5 min). `--prod` requires typing `WRITE PRODUCTION`; `--session <minutes>` (≤ 120) opens a window; `--end-session` revokes; `--apex-file` for Apex; `--impact --sobject X --where "…"` binds the token to a verified record count, and the gate refuses later if the criteria come to match more. Operator TTY only.
`blast-radius`	What an operation will set off — scope, triggers, flows, validation rules, roll-ups, cascade deletes and the lookup children left orphaned. Read-only. Reports UNDETERMINED rather than a silent zero.
`checkup <org>`	Runs every catalogue probe against the org — which traps are live here, each citing the entry that predicted it. `--record` stores the findings against that org.
`lesson`	`fact` (schema-enforced platform entry) · `gate` (permanent fixture) · `org` (a finding about one org) · `review` (what the observer captured, ready to record).
`install-gates`	Register the hooks at user level so they bind outside this workspace.
`log`	Append a redacted session entry with before → after values.
`attest`	Emit machine-readable attestations of what was verified.
`frontdoor`	Authenticated Lightning session for browser verification — writes the URL to a 0600 file and prints only the path.
`harness/validate.py`	`--profile static|capability|release` · `--self-test` · `--target-org` · `--only`

LAYOUT

`.claude/skills/`	The five operations, each a documented sequence
`.claude/rules/`	The seven rule files, byte-capped, each labelled with what backs it
`.claude/agents/`	org-explorer (read-only) · hostile-qa (adversarial reviewer)
`hooks/`	The shared expansion-aware parser and the two gates
`harness/`	120 checks, 260 gate fixtures (257 recorded + 3 minted live), 19 mutators, live_matrix.sh, the run log
`SECURITY.md`	Disclosure policy — 72-hour confirmation, 90-day ceiling, fixtures credited

Torque is open source under MIT, and it is meant to be run rather than read. Clone it, point it at a free Developer Edition org, and watch it refuse things.

Omid Mojtahedi · omid.mojtahedi@gmail.com
github.com/omoji-personal/torque